

UNIT 2

SOLVING PROBLEMS BY SEARCHING

Problem Solving Agents

Problem-solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Search algorithm

A search algorithm is **the step-by-step procedure used to locate specific data among a collection of data**. It is considered a fundamental procedure in computing. In computer science, when searching for data, the difference between a fast application and a slower one often lies in the use of the proper search algorithm

Search Algorithm Terminologies

1. **Search:** Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
 - a. **Search Space:** Search space represents a set of possible solutions, which a system may have.
 - b. **Start State:** It is a state from where agent begins **the search**.
 - c. **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.
2. **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
3. **Actions:** It gives the description of all the available actions to the agent.
4. **Transition model:** A description of what each action do, can be represented as a transition model.
5. **Path Cost:** It is a function which assigns a numeric cost to each path.
6. **Solution:** It is an action sequence which leads from the start node to the goal node.
7. **Optimal Solution:** If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Steps problem-solving in AI

The problem of AI is directly associated with the nature of humans and their activities. So we need a number of finite steps to solve a problem which makes human easy works.

These are the following steps which require to solve a problem :

- **Goal Formulation:** This one is the first and simple step in problem-solving. It organizes finite steps to formulate a target/goals which require some action to achieve the goal. Today the formulation of the goal is based on AI agents.
- **Problem formulation:** It is one of the core steps of problem-solving which decides what action should be taken to achieve the formulated goal. In AI this core part is dependent upon software agent which consisted of the following components to formulate the associated problem.

Components to formulate the associated problem:

- **Initial State:** This state requires an initial state for the problem which starts the AI agent towards a specified goal. In this state new methods also initialize problem domain solving by a specific class.
- **Action:** This stage of problem formulation works with function with a specific class taken from the initial state and all possible actions done in this stage.
- **Transition:** This stage of problem formulation integrates the actual action done by the previous action stage and collects the final stage to forward it to their next stage.
- **Goal test:** This stage determines that the specified goal achieved by the integrated transition model or not, whenever the goal achieves stop the action and forward into the next stage to determines the cost to achieve the goal.
- **Path costing:** This component of problem-solving numerical assigned what will be the cost to achieve the goal. It requires all hardware software and human working cos

EXAMPLE PROBLEMS

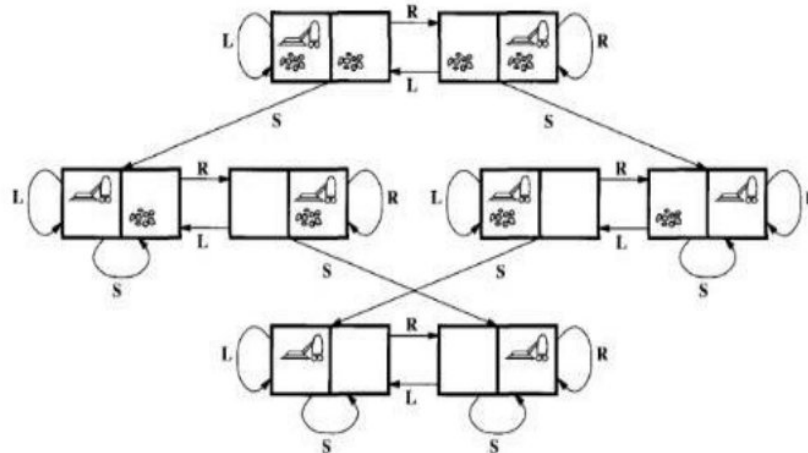
- a. The problem solving approach has been applied to a vast array of task environments.
 - i. Some best known problems are summarized below.
- b. They are distinguished as toy or real-world problems
 1. A **Toy problem** is intended to illustrate various problem solving ~~mtok~~ It can be easily used by different researchers to compare the performance of algorithms.
 2. A **Real world problem** is one whose solutions people actually care about.

TOY PROBLEMS

Vacuum World Example

1. **States:** The agent is in one of two locations, each of which might or might not contain dirt. Thus there are $2 \times 2^2 = 8$ possible world states.
2. **Initial state:** Any state can be designated as initial state.
3. **Successor function :** This generates the legal states that results from trying the three actions (left, right, suck). The complete state space is shown in figure 2.3
4. **Goal Test :** This tests whether all the squares are clean.
5. **Path test :** Each step costs one ,so that the the path cost is the number of steps in the path.

Vacuum World State Space

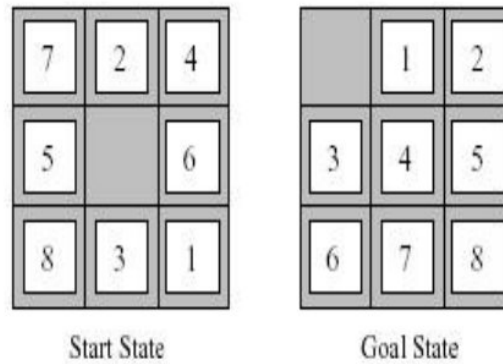


The state space for the vacuum world.
Arcs denote actions: L = Left, R = Right, S = Suck

8-puzzle:

1. An 8-puzzle consists of a 3x3 board with eight numbered tiles and a blank space.
2. A tile adjacent to the blank space can slide into the space. The object is to reach the specific goal state ,as shown in figure

Example: The 8-puzzle

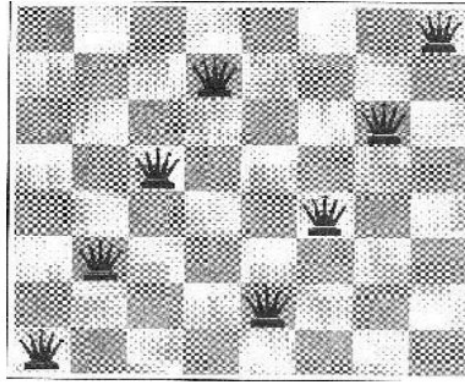


The problem formulation is as follows :

- a. **States** : A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- b. **Initial state** : Any state can be designated as the initial state. It can be noted that any given goal can be reached from exactly half of the possible initial states.
- c. **Successor function** : This generates the legal states that result from trying the four actions (blank moves Left, Right, Up or down).
- d. **Goal Test** : This checks whether the state matches the goal configuration shown in figure 2.4. (Other goal configurations are possible)
- e. **Path cost**: Each step costs 1, so the path cost is the number of steps in the path.
- f. **The 8-puzzle** belongs to the family of **sliding-block puzzles**, which are often used as
 - i. test problems for new search algorithms in AI.
- g. This general class is known as NP-complete.
- h. The **8-puzzle** has $9!/2 = 181,440$ reachable states and is easily solved.
- i. The **15 puzzle** (4 x 4 board) has around 1.3 trillion states, and the random instances can be solved optimally in few milli seconds by the best search algorithms.
- j. The **24-puzzle** (on a 5 x 5 board) has around 10^{25} states, and random instances are still quite difficult to solve optimally with current machines and algorithms.

8-queens problem

1. The goal of 8-queens problem is to place 8 queens on the chessboard such that no queen attacks any other. (A queen attacks any piece in the same row, column or diagonal).
2. The following figure shows an attempted solution that fails: the queen in the right most column is attacked by the queen at the top left.
3. An **Incremental formulation** involves operators that augments the state description, starting with an empty state. for 8-queens problem, this means each action adds a queen to the state.
4. A **complete-state formulation** starts with all 8 queens on the board and move them around.
5. In either case the path cost is of no interest because only the final state counts.



8-queens problem

The first incremental formulation one might try is the following :

- a. **States** : Any arrangement of 0 to 8 queens on board is a state.
- b. **Initial state** : No queen on the board.
- c. **Successor function** : Add a queen to any empty square.
- d. **Goal Test** : 8 queens are on the board, none attacked.

In this formulation, we have $64 \cdot 63 \cdot \dots \cdot 57 = 3 \times 10^{14}$ possible sequences to investigate.

A better formulation would prohibit placing a queen in any square that is already attacked. :

- a. **States** : Arrangements of n queens ($0 \leq n \leq 8$), one per column in the left most columns, with no queen attacking another are states.
- b. **Successor function** : Add a queen to any square in the left most empty column such that it is not attacked by any other queen.

This formulation reduces the 8-queen state space from 3×10^{14} to just 2057, and solutions are easy to find.

- a. For the 100 queens the initial formulation has roughly 10^{400} states whereas the improved formulation has about 10^{52} states.
- b. This is a huge reduction, but the improved state space is still too big for the algorithms to handle.

REAL WORLD PROBLEMS

1. A real world problem is one whose solutions people actually care about.
2. They tend not to have a single agreed upon description, but attempt is made to give general flavor of their formulation,

The following are the some real world problems,

- a. Route Finding Problem
- b. Touring Problems
- c. Travelling Salesman Problem
- d. Robot Navigation

ROUTE-FINDING PROBLEM

1. Route-finding problem is defined in terms of specified locations and transitions along links between them.
2. Route-finding algorithms are used in a variety of applications, such as routing in computer networks, military operations planning, and air line travel planning systems.

AIRLINE TRAVEL PROBLEM

The **airline travel problem** is specifies as follows :

1. **States** : Each is represented by a location(e.g.,an airport) and the current time.
2. **Initial state** : This is specified by the problem.
3. **Successor function** : This returns the states resulting from taking any scheduled flight(further specified by seat class and location),leaving later than the current time plus the within-airport transit time,from the current airport to another.
4. **Goal Test** : Are we at the destination by some prespecified time?
5. **Path cost** : This depends upon the monetary cost,waiting time,flight time,customs and immigration procedures,seat quality,time of dat,type of air plane,frequent-flyer mileage awards, and so on.

TOURING PROBLEMS

1. **Touring problems** are closely related to route-finding problems,but with an important difference.
2. Consider for example, the problem, "Visit every city at least once" as shown in Romania map.
3. As with route-finding the actions correspond to trips between adjacent cities. The state space, however,is quite different.
4. **Initial state** would be "In Bucharest; visited{ Bucharest}".
5. **Intermediate state** would be "In Vaslui; visited
a. { Bucharest,Vrziceni,Vaslui}".
6. **Goal test** would check whether the agent is in Bucharest and all cities have been visited.

THE TRAVELLING SALESPERSON PROBLEM (TSP)

1. TSP is a touring problem in which each city must be visited exactly once.
2. The aim is to find the shortest tour. The problem is known to be NP-hard.
3. Enormous efforts have been expended to improve the capabilities of TSP algorithms.
4. These algorithms are also used in tasks such as planning movements of **automatic circuit-board drills** and of **stocking machines** on shop floors.

VLSI layout

A **VLSI layout** problem requires positioning millions of components and connections on a chip to minimize area ,minimize circuit delays,minimize stray capacitances,and maximize manufacturing yield. The layout problem is split into two parts : **cell layout** and **channel routing**.

ROBOT NAVIGATION

ROBOT navigation is a generalization of the route-finding problem. Rather than a discrete set of routes, a robot can move in a continuous space with an infinite set of possible actions and states. For a circular Robot moving on a flat surface, the space is essentially two-dimensional.

When the robot has arms and legs or wheels that also must be controlled, the search space becomes multi-dimensional. Advanced techniques are required to make the search space finite.

AUTOMATIC ASSEMBLY SEQUENCING

The example includes assembly of intricate objects such as electric motors. The aim in assembly problems is to find the order in which to assemble the parts of some objects. If the wrong order is chosen, there will be no way to add some part later without undoing some work already done.

Another important assembly problem is protein design, in which the goal is to find a sequence of Amino acids that will be fold into a three-dimensional protein with the right properties to cure some disease.

INTERNET SEARCHING

In recent years there has been increased demand for software robots that perform Internet searching, looking for answers to questions, for related information, or for shopping deals.

The searching techniques consider internet as a graph of nodes (pages) connected by links.

MEASURING PROBLEM-SOLVING PERFORMANCE

The output of problem-solving algorithm is either failure or a solution. (Some algorithms might stuck in an infinite loop and never return an output.)

The algorithm's performance can be measured in four ways :

- a. **Completeness:** Is the algorithm guaranteed to find a solution when there is one?
- b. **Optimality :** Does the strategy find the optimal solution
- c. **Time complexity:** How long does it take to find a solution?
- d. **Space complexity:** How much memory is needed to perform the search

SEARCHING FOR SOLUTIONS

State Spaces

One general formulation of intelligent action is in terms of a state space. A state contains all of the information necessary to predict the effects of an action and to determine whether a state satisfies the goal.

Assumptions of State-space searching :

1. The agent has perfect knowledge of the state space and is planning for the case where it observes what state it is in: there is full observability.
2. The agent has a set of actions that have known deterministic effects.
3. The agent can determine whether a state satisfies the goal

A **state-space problem** consists of

1. A set of states
2. A distinguished state called the **start state**
3. For each state, a set of actions available to the agent in that state
4. an **action function** that, given a state and an action, returns a new state
5. A **goal** specified as a Boolean function, $goal(s)$, that is true when state s satisfies the goal, in which case we say that s is a **goal state**
6. A criterion that specifies the quality of an acceptable solution. For example, any sequence of actions that gets the agent to the goal state may be acceptable, or there may be costs associated with actions and the agent may be required to find a sequence that has minimal total cost. A solution that is best according to some criterion is called an **optimal solution**. We do not always need an optimal solution, for example, we may be satisfied with any solution that is within 10% of optimal.

Graph Searching

1. The problem of finding a sequence of actions to achieve a goal is abstracted as searching for paths in directed graphs.
2. To solve a problem, first define the underlying search space and then apply a search algorithm to that search space.
3. Many problem-solving tasks are transformable into the problem of finding a path in a graph.
4. Searching in graphs provides an appropriate abstract model of problem solving independent of a particular domain.

A Generic Searching Algorithm

A generic algorithm to search for a solution path in a graph. The algorithm calls procedures that can be coded to implement various search strategies.

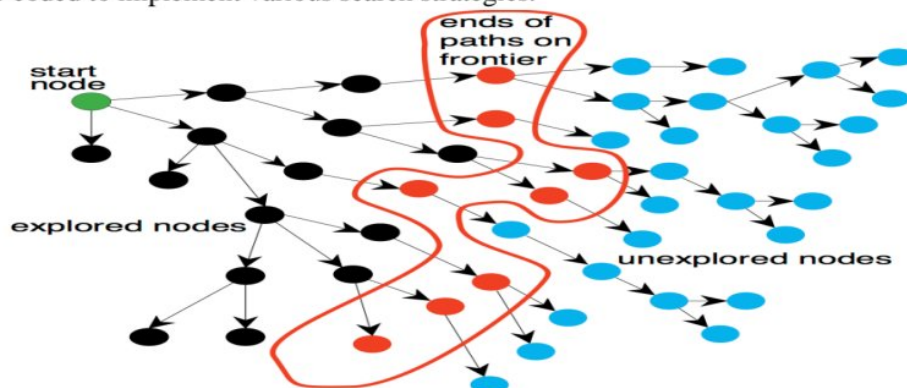


FIG: Problem solving by graph searching

The intuitive idea behind the generic search algorithm, given a graph, a start node, and a goal predicate, is to explore paths incrementally from the start node. This is done by maintaining a **frontier** (or **fringe**) of paths from the start node. The frontier contains all of the paths that could form initial segments of paths from the start node to a goal node. Initially, the frontier contains the trivial path containing just the start node, and no arcs. As the search proceeds, the frontier expands into the unexplored nodes until a goal node is encountered. Different search strategies are obtained by providing an appropriate implementation of the frontier.

Generic graph searching algorithm

```

1. procedure Search(G,S,goalG,S,goal)
2:   Inputs
3:     GG: graph with nodes NN and arcs AA
4:     ss: start node
5:     goal: Boolean function of nodes
6:   Output
7:     path from ss to a node for which goal is true
8:     or  $\perp$  if there are no solution paths
9:   Local
10:    Frontier: set of paths
11:    Frontier := { {s} }
12:    while Frontier  $\neq \{\}$  do
13:      select and remove  $\langle n_0, \dots, n_k \rangle$  from Frontier
14:      if goal( $n_k$ ) then
15:        return  $\langle n_0, \dots, n_k \rangle$ 
16:      Frontier := Frontier  $\cup \{ \langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A \}$ 
17:    return  $\perp$ 

```

The frontier is a set of paths. Initially, the frontier contains the path of zero cost consisting of just the start node. At each step, the algorithm removes a path $\langle n_0, \dots, n_k \rangle$ from the frontier. If goal(n_k) is true (i.e., n_k is a goal node), it has **found a solution** and returns the path $\langle n_0, \dots, n_k \rangle$. Otherwise, the path is extended by one more arc by finding the neighbors of n_k . For every neighbor n of n_k , the path $\langle n_0, \dots, n_k, n \rangle$ is added to the frontier. This step is known as **expanding** the path $\langle n_0, \dots, n_k \rangle$.

This algorithm has a few features that should be noted:

1. Which path is selected at line 13 defines the search strategy. The selection of a path can affect the efficiency; see the box for more details on the use of “select”.
2. It is useful to think of the **return** at line 15 as a temporary return, where a caller can **retry** the search to get another answer by continuing the while loop. This can be implemented by having a class that keeps the state of the search and a search() method that returns the next solution.

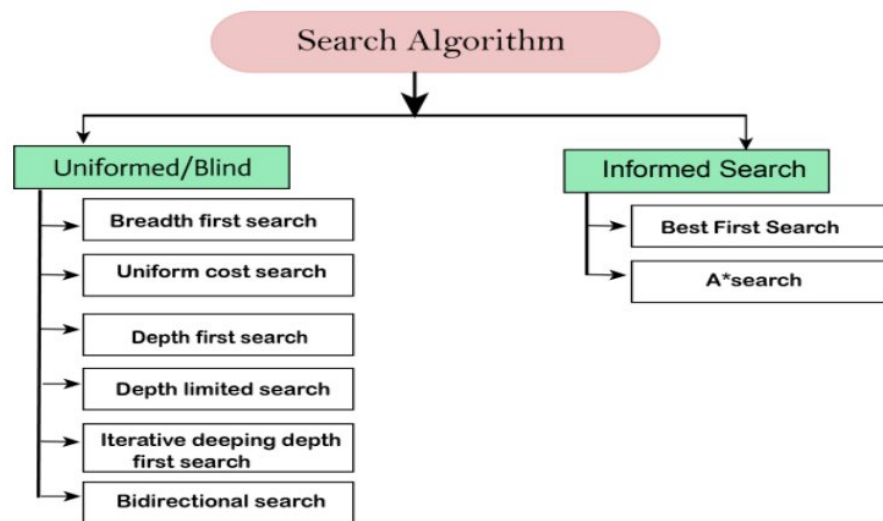
3.If the procedure returns $\perp\perp$ (“**bottom**”), there are no solutions, or no remaining solutions if the search has been retried.

4.The algorithm only tests if a path ends in a goal node *after* the path has been selected from the frontier, not when it is added to the frontier. There are two important reasons for this. There could be a costly arc from a node on the frontier to a goal node. The search should not always return the path with this arc, because a lower-cost solution may exist. This is crucial when the lowest-cost path is required. A second reason is that it may be expensive to determine whether a node is a goal node, and so this should be delayed in case the computation is not necessary.

If the node at the end of the selected path is not a goal node and it has no neighbors, then extending the path means removing the path from the frontier. This outcome is reasonable because this path could not be part of a path from the start node to a goal node.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

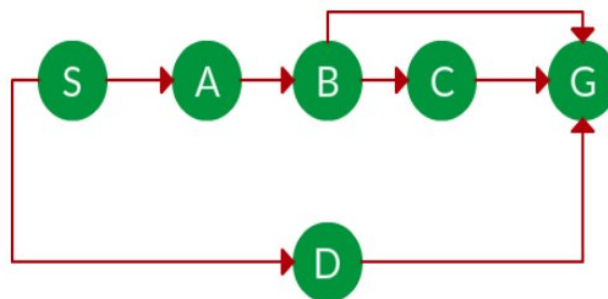
It can be divided into five main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

Breadth-first search

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbour nodes at the present depth prior to moving on to the nodes at the next depth level.

BFS expands the leaf node with the lowest path cost so far, and keeps going until a goal node is generated. If the path cost simply equals the number of links, we can implement this as a simple queue ("first in, first out").



Depth-first Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **$O(bm)$** .

Depth-Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Bidirectional search

Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

Disadvantages:

Implementation of the bidirectional search tree is difficult.

In bidirectional search, one should know the goal state in advance.

Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is $O(b^d)$.

Space Complexity: Space complexity of bidirectional search is $O(b^d)$.

Optimal: Bidirectional search is Optimal.

Uniform-cost Search Algorithm:

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs from the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Advantages:

Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

It does not care about the number of steps involved in searching and is only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Best first search

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e. $f(n) = g(n)$.

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- **Step 4:** Expand the node n , and generate the successors of node n .
- **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

Iterative deepening algorithm

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Completeness: This algorithm is complete if the branching factor is finite.

Time Complexity: Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity: The space complexity of IDDFS will be $O(bd)$.

Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function:

Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$. It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

Best First Search Algorithm(Greedy search)

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first

search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$f(n) = g(n) + h(n)$$

Where, $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- **Step 4:** Expand the node n , and generate the successors of node n .
- **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Advantages:

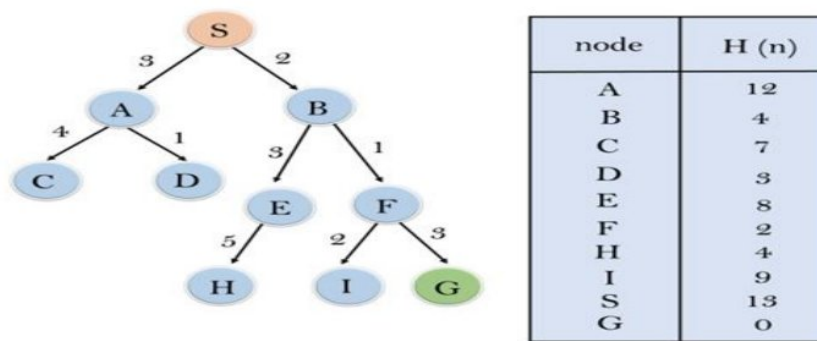
- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

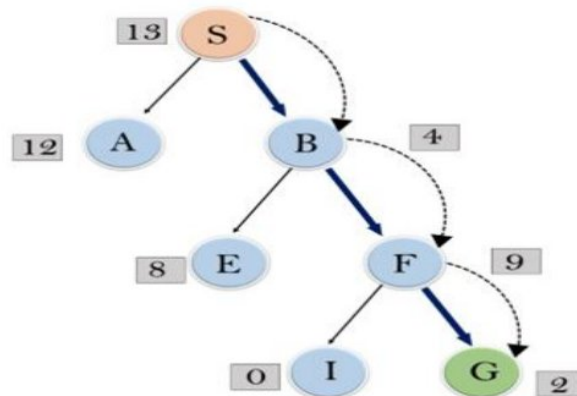
- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B, F]
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S----> B----->F----> G**

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

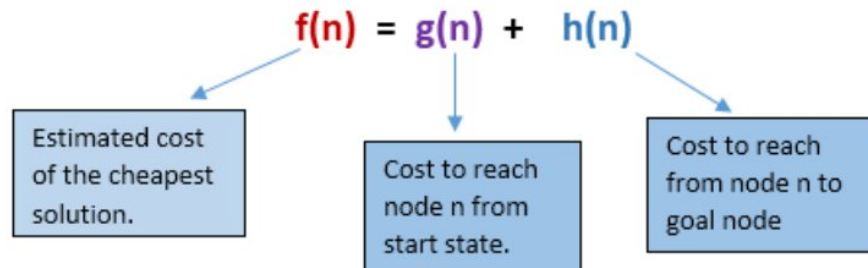
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

2.) A* Search Algorithm:

A* search is the most commonly known form of best-first search. It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$. It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.



Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.

Step 6: Return to Step 2.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

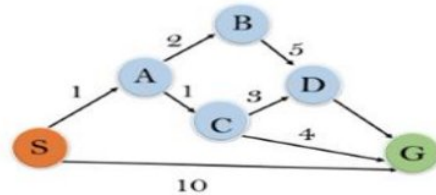
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

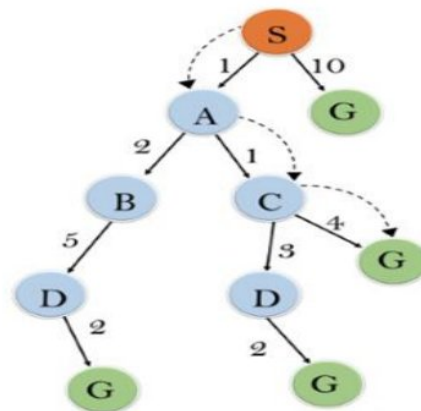
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: $\{(S, 5)\}$

Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$

Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration 4 will give the final result, as $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition $f(n) \leq l_i$

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So the time complexity is $O(b^d)$, where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is $O(b^d)$

Recursive Best-first Search (RBFS)

1. Recursive best-first search is a simple recursive algorithm that attempts to mimic the operation of standard best-first search, but using only linear space.
2. The algorithm is shown in below figure.
3. Its structure is similar to that of recursive depth-first search, but rather than continuing indefinitely down the current path, it keeps track of the f -value of the best alternative path available from any ancestor of the current node.
4. If the current node exceeds this limit, the recursion unwinds back to the alternative path. As the recursion unwinds, RBFS replaces the f -value of each node along the path with the best f -value of its children.

function RECURSIVE-BEST-FIRST-SEARCH(problem) **return** a solution or failure
return RFBS(problem, MAKE-NODE(INITIAL-STATE[problem]), ∞)

function RFBS(problem, node, f_limit) **return** a solution or failure and a new f -cost limit

if GOAL-TEST[problem](STATE[node]) **then return** node successors

□ EXPAND(node, problem)

if successors is empty **then return** failure, ∞

for each s **in** successors **do**

$f[s]$ □ $\max(g(s) + h(s), f[node])$

repeat

best □ the lowest f -value node in successors

if $f[best] > f_limit$ **then return** failure, $f[best]$

alternative □ the second lowest f -value among successors result, f

$[best]$ □ RFBS(problem, best, $\min(f_limit, alternative)$) **if** result □

failure **then return** result

The algorithm for recursive best-first search

RBFS Evaluation:

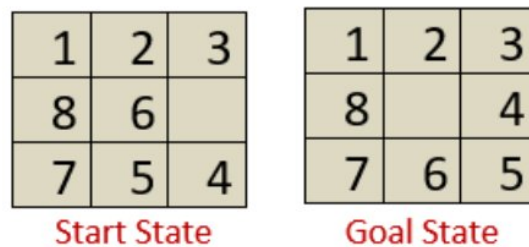
- a. RBFS is a bit more efficient than IDA*
 - i. Still excessive node generation (mind changes)
- b. Like A*, optimal if $h(n)$ is admissible
- c. Space complexity is $O(bd)$.
 - i. IDA* retains only one single number (the current f-cost limit)
- d. Time complexity difficult to characterize
 - i. Depends on accuracy of $h(n)$ and how often best path changes.
- e. IDA* and RBFS suffer from *too little* memory.

Heuristic Functions

A **heuristic function** or simply a heuristic is a function that ranks alternatives in various search algorithms at each branching step basing on an available information in order to make a decision which branch is to be followed during a search

Example: 8 Puzzle

Consider the following 8-puzzle problem where we have a start state and a goal state. Our task is to slide the tiles of the current/start state and place it in an order followed in the goal state. There can be four moves either **left, right, up, or down**. There can be several ways to convert the current/start state to the goal state, but, we can use a heuristic function $h(n)$ to solve the problem more efficiently.

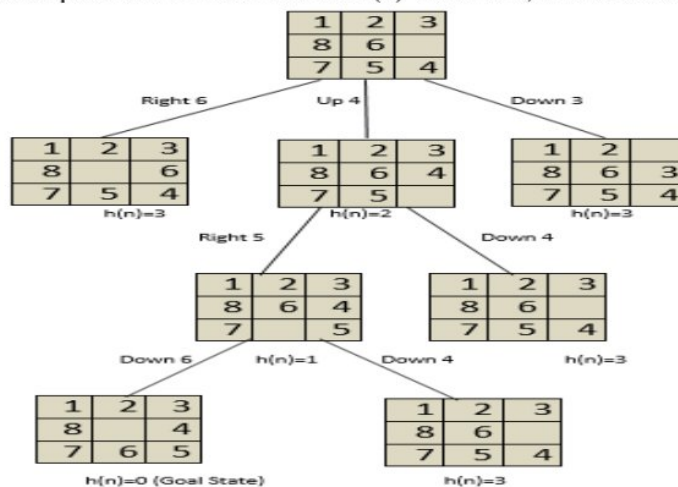


A heuristic function for the 8-puzzle problem is defined below:

$h(n)$ = Number of tiles out of position.

So, there is total of three tiles out of position i.e., 6, 5 and 4. Do not count the empty tile present in the goal state). i.e. $h(n)=3$. Now, we require to minimize the value of **$h(n) = 0$** .

We can construct a state-space tree to minimize the $h(n)$ value to 0, as shown below:



It is seen from the above state space tree that the goal state is minimized from $h(n)=3$ to $h(n)=0$. However, we can create and use several heuristic functions as per the requirement. It is also clear from the above example that a heuristic function $h(n)$ can be defined as the information required to solve a given problem more efficiently. The information can be related to the **nature of the state, cost of transforming from one state to another, goal node characteristics**, etc., which is expressed as a heuristic function.

Properties of a Heuristic search Algorithm

Use of heuristic function in a heuristic search algorithm leads to following properties of a heuristic search algorithm:

- **Admissible Condition:** An algorithm is said to be admissible, if it returns an optimal solution.
- **Completeness:** An algorithm is said to be complete, if it terminates with a solution (if the solution exists).
- **Dominance Property:** If there are two admissible heuristic algorithms **A1** and **A2** having **h1** and **h2** heuristic functions, then **A1** is said to dominate **A2** if **h1** is better than **h2** for all the values of node **n**.
- **Optimality Property:** If an algorithm is **complete, admissible**, and **dominating** other algorithms, it will be the best one and will definitely give an optimal solution.

The Effective Branching factor

- a. One way to characterize the **quality of a heuristic** is the **effective branching factor b^*** .
If the total number of nodes generated by A^* for a particular problem is N , and the **solution depth** is d , then b^* is the branching factor that a uniform tree of depth d would have to have in order to contain $N+1$ nodes. Thus,

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$
- b. For example, if A^* finds a solution at depth 5 using 52 nodes, then effective branching factor is 1.92.
- c. A well designed heuristic would have a value of b^* close to 1, allowing fairly large problems to be solved.
- d. To test the heuristic functions h_1 and h_2 , 1200 random problems were generated with solution lengths from 2 to 24 and solved them with iterative deepening search and with A^* search using both h_1 and h_2 .
- e. The following table gives the average number of nodes expanded by each strategy and
 - i. the effective branching factor.
- f. The results suggest that h_2 is better than h_1 and is far better than using iterative deepening search.
- g. For a solution length of 14, A^* with h_2 is 30,000 times more efficient than uninformed iterative deepening search.

d	Search Cost			Effective Branching Factor		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

Comparison of search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* Algorithms with h_1 and h_2 . Data are average over 100 instances of the 8-puzzle, for various solution lengths.

Hill Climbing Algorithm in Artificial Intelligence

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

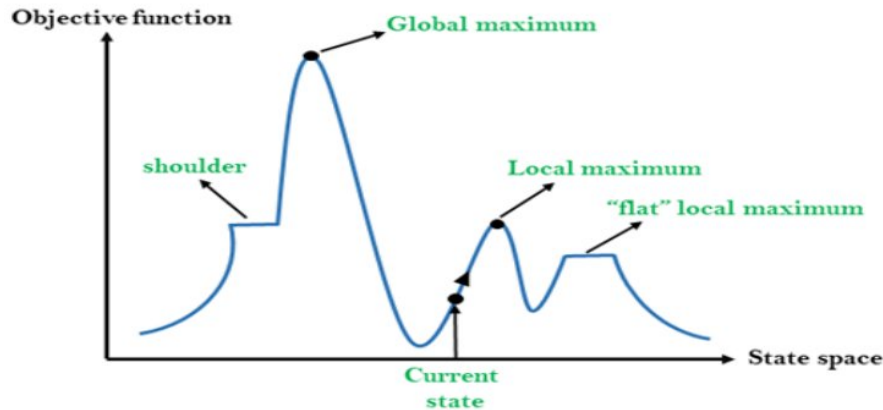
Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present.

Flat local maximum: It is a flat space in the landscape where all the neighbor states of current states have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state.** It only checks its one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:

- Less time consuming

- Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
 1. If it is goal state, then return success and quit.
 2. Else if it is better than the current state then assign new state as a current state.
 3. Else if not better than the current state, then return to step2.
- **Step 5:** Exit.

2. Steepest-Ascent hill climbing:

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors

Algorithm for Steepest-Ascent hill climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- **Step 2:** Loop until a solution is found or the current state does not change.
 1. Let SUCC be a state such that any successor of the current state will be better than it.
 2. For each operator that applies to the current state:
 - I. Apply the new operator and generate a new state.
 - II. Evaluate the new state.
 - III. If it is goal state, then return it and quit, else compare it to the SUCC.
 - IV. If it is better than SUCC, then set new state as SUCC.
 - V. If the SUCC is better than the current state, then set current state to SUCC.
- **Step 5:** Exit.

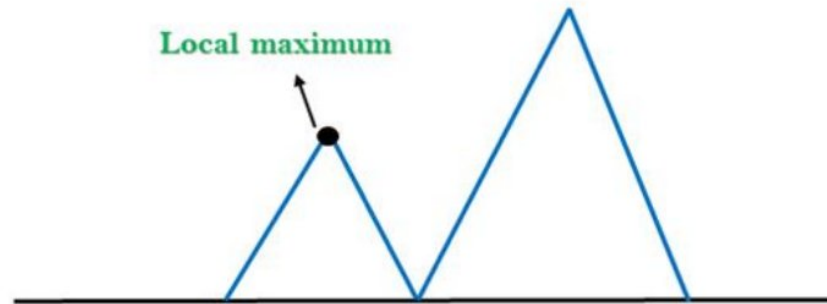
3. Stochastic hill climbing:

Stochastic hill climbing does not examine for all its neighbor before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Problems in Hill Climbing Algorithm:

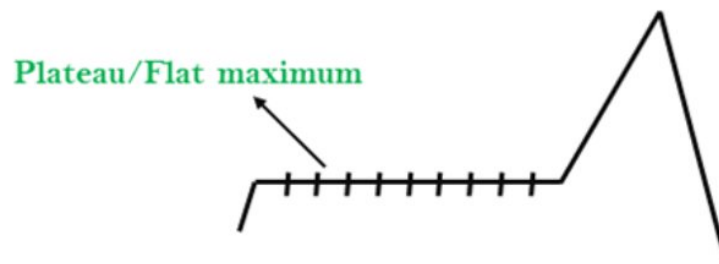
1. Local Maximum: A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

Solution: Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.



2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.



Simulated Annealing:

A hill-climbing algorithm which never makes a move towards a lower value guaranteed to be incomplete because it can get stuck on a local maximum. And if algorithm applies a random walk, by moving a successor, then it may complete but not efficient. **Simulated Annealing** is an algorithm which yields both efficiency and completeness.

In mechanical term **Annealing** is a process of hardening a metal or glass to a high temperature then cooling gradually, so this allows the metal to reach a low-energy crystalline state. The same process is used in simulated annealing in which the algorithm picks a random move, instead of picking the best move. If the random move improves the state, then it follows the same path. Otherwise, the algorithm follows the path which has a probability of less than 1 or it moves downhill and chooses another path.

Inventing admissible heuristic functions

Relaxed problems

- A problem with fewer restrictions on the actions is called a **relaxed problem**
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move *anywhere*, then $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to *any adjacent square*, then $h_2(n)$ gives the shortest solution

Local Search Algorithms and Optimization Problem

The informed and uninformed search expands the nodes systematically in two ways:

- keeping different paths in the memory and
- selecting the best suitable path,

Which leads to a solution state required to reach the goal node. But beyond these “classical search algorithms,” we have some “local search algorithms” where the path cost does not matters, and only focus on solution-state needed to reach the goal node.

A local search algorithm completes its task by traversing on a single current node rather than multiple paths and following the neighbors of that node generally.

Although local search algorithms are not systematic, still they have the following two advantages:

- Local search algorithms use a very little or constant amount of memory as they operate only on a single path.
- Most often, they find a reasonable solution in large or infinite state spaces where the classical or systematic algorithms do not work.

PURE OPTIMIZED PROBLEM

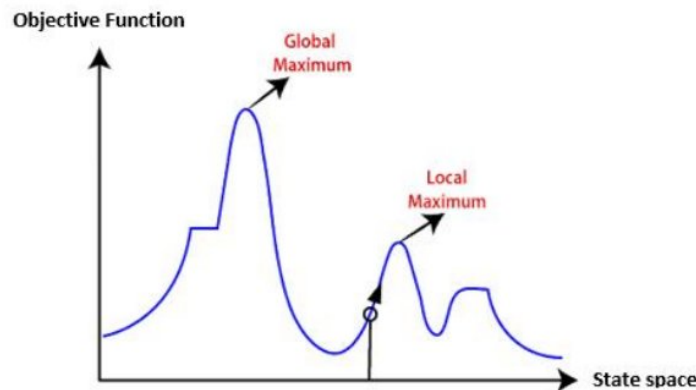
A pure optimization problem is one where all the nodes can give a solution. But the target is to find the best state out of all according to the **objective function**. Unfortunately, the pure optimization problem fails to find high-quality solutions to reach the goal state from the current state.

An objective function is a function whose value is either minimized or maximized in different contexts of the optimization problems. In the case of search algorithms, an objective function can be the path cost for reaching the goal node, etc.

Working of a Local search algorithm

Consider the below state-space landscape having both:

- **Location:** It is defined by the state.
- **Elevation:** It is defined by the value of the objective function or heuristic cost function.



A one-dimensional state-space landscape in which elevation corresponds to the objective function

The local search algorithm explores the above landscape by finding the following two points:

- **Global Minimum:** If the elevation corresponds to the cost, then the task is to find the lowest valley, which is known as **Global Minimum**.
- **Global Maxima:** If the elevation corresponds to an objective function, then it finds the highest peak which is called as **Global Maxima**. It is the highest point in the valley.

We will understand the working of these points better in Hill-climbing search.

Below are some different types of local searches:

- Hill-climbing Search
- Simulated Annealing
- Local Beam Search

Online search agents

Online search is a necessary idea for unknown environments, where the agent does not know what states exist or what its actions do. In this state of ignorance, the agent faces an exploration problem and must use its actions as experiments in order to learn enough to make deliberation

The canonical example of online search is a robot that is placed in a new building and must explore it to build a map that it can use for getting from A to B. Methods for escaping from labyrinths—required knowledge for aspiring heroes of antiquity—are also examples of online search algorithms

Spatial exploration is not the only form of exploration, however. Consider a newborn baby: it has many possible actions but knows the outcomes of none of them, and it has experienced only a few of the possible states that it can reach. The baby's gradual discovery of how the world works is, in part, an online search process.

Online search problems

An online search problem must be solved by an agent executing actions, rather than by pure computation. We assume a deterministic and fully observable environment (Chapter 17 relaxes these assumptions), but we stipulate that the agent knows only the following:

- $ACTIONS(s)$, which returns a list of actions allowed in state s ;
- The step-cost function $c(s, a, s')$ —note that this cannot be used until the agent knows that s' is the outcome; and
- $GOAL-TEST(s)$.

Note in particular that the agent cannot determine $RESULT(s, a)$ except by actually being in s and doing a . For example, in the maze problem shown in Figure 4.19, the agent does not know that going Up from (1,1) leads to (1,2); nor, having done that, does it know that going Down will take it back to (1,1). This degree of ignorance can be reduced in some applications—for example, a robot explorer might know how its movement actions work and be ignorant only of the locations of obstacles